Reviewer Pack — Submodular Width of Polymatroid Rank Function and Monotone C...

Entry 9efd21df5cc8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 09:58:39 UTC

Submodular Width of Polymatroid Rank Function and Monotone Circuit Size

Entry ID: `9efd21df5cc8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 09:58:39 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

> For a k-CLIQUE CNF instance Φ, the submodular width of its associated polymatroid P_Φ is Ω(n), while for general CNFs, it is O(log n).

Rationale (proposer's reasoning):

> The submodular width of P_Φ captures the combinatorial structure of Φ that makes it resistant to efficient monotone circuit computation, mirroring the inherent complexity of k-CLIQUE.

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `22550f86323dec71`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=241.5s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def polymatroid_rank(X):
        if not X:
            return 0
        n = len(X)
        P = [1] * (1 << n)
        for i in range(1, 1 << n):
            for j in range(i):
                if (i & j) == j and (i ^ j).bit_count() == 1:
                    P[i] += P[j]
        return sum(P[i] for i in range(len(X)) if (X >> i) & 1)

    def submodular_width(n, X, Y):
        return polymatroid_rank(X) + polymatroid_rank(Y) - polymatroid_rank(X & Y)

    def generate_k_clique_instance(n, k):
        edges = []
        for i in range(k):
            for j in range(i+1, k):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def generate_general_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = set()
            for i in range(n):
                if random.random() < 0.5:
                    clause.add(i)
            clauses.append(clause)
        return clauses

    n = random.randint(5, 40)
    k = random.randint(1, min(n-1, 3))

    if random.random() < 0.5:
        instance_type = "k-clique"
        instance = generate_k_clique_instance(n, k)
    else:
```

```
        instance_type = "general"
        instance = generate_general_cnf(n)

    P_clique = [polymatroid_rank([i for i in range(n)])]
    for X in range(1 << n):
        P_clique.append(polymatroid_rank(X))

    max_width = 0
    for X in range(1 << n):
        for Y in range(1 << n):
            width = submodular_width(n, X, Y)
            if width > max_width:
                max_width = width

    if instance_type == "k-clique":
        lower_bound = math.ceil(n ** (k / 4))
        upper_bound = float('inf')
    else:
        lower_bound = 0
        upper_bound = n * math.log2(n)

    return {
        "metric_name": "submodular_width",
        "metric_value": max_width,
        "instances_tested": 1,
        "conjecture_holds": lower_bound <= max_width <= upper_bound,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test timed out without producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and debug crash cause

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	71327
2	preregistration	ollama_remote	qwen3:8b	0	0	24022
3	novelty	ollama_remote	qwen3:8b	0	0	20615
4	novelty	ollama_remote	qwen3:8b	0	0	13956
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19234
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10170
7	judge	ollama_remote	qwen3:8b	0	0	143054

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 302377 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/9efd21df5cc8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9efd21df5cc8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9efd21df5cc8.tar.gz` (if generated)